Progress Report 1

Selective State Recovery for Reliable AI Agents

Oludolapo Adegbesan
Case study: Fisk University B.S. in Computer Science

1. Project Overview

My project investigates how a tool-using AI agent can recover when an early mistake becomes visible later in a workflow. The central question is how to identify a safe recovery boundary, invalidate affected results, and preserve valid independent work. Academic planning is the intended case study because credits, grades and requirements provide opportunities for deterministic validation.

I am developing and evaluating a selective recovery prototype. Its effectiveness remains a research question. Published benchmark components provide a reference environment; custom recovery modifications are labeled separately.

2. Work Completed

I implemented a Python execution-trace and dependency layer, a deterministic recovery demonstration, and a live Gemini diagnosis harness. The synthetic comparison includes full restart, visible-step retry, simulated late-checkpoint restoration, source rollback, and selective recovery.

I integrated a Research Lab into the Next.js/FastAPI application. It launches fixed experiments and displays saved results, methods, traces, provenance and JSON exports. The interface separates official benchmarks, custom recovery on official tasks, synthetic demonstrations, and the Fisk catalog case study.

I pinned official AgentRx and maintained tau benchmark repositories, ran official preprocessing and environment checks, and executed a live released retail task. I then implemented an external controlled-fault recovery adapter using the released tools and unchanged evaluator. I also extracted Fisk catalog facts and created four synthetic academic audit cases.

> **Current status:** a working research prototype with preliminary live evidence. Full published recovery-method reproduction, repeated experiments and complete Fisk graduation validation remain unfinished.

GitHub: https://github.com/Havcker243/plan-my-path
This is the configured repository remote. Local changes have not been verified as pushed or publicly accessible.

3. Evidence of Progress

Official components. The pinned maintained tau repository supplies retail tasks, policy, a simulated database, agent/user implementation and grading [1]. Gemini replaces upstream model defaults, so these runs do not reproduce published aggregate scores. One unchanged task-0 smoke test earned reward **0.0**; that result is preserved separately.

Custom recovery pilot. On October 7, I completed a matched pair on released retail task 0, seed 300. Once per execution, the first successful user-details read was replaced with an empty object. Both strategies used Gemini 3.5 Flash Lite for agent and user, temperature 0, six-second generation pacing, a 60-attempt-step limit and a 180-second wall-time budget. At most one automatic recovery was allowed.

Observed metric	No added recovery	Selective recovery
Official reward	1.0	1.0
Attempt steps	21	25
Model calls	15	17
Task tool executions	6	8
Validation tool calls	9	9
Cached reads reused	0	0
Estimated model cost (USD)	$0.02614	$0.02801
Duration including pacing (s)	127.20	128.76

Table 1. Completed live pair. Counts include discarded attempts; validation tool calls are reported separately. Source: paced-pilot recovery-results.json, October 7.

Both runs reached the injected fault and earned official reward **1.0**. The shared detector matched the injected source in both executions. The reference agent recovered without added rollback. Selective recovery restored the earlier boundary but used more work. **This pair shows no reward or efficiency advantage for selective recovery.**

> This is a researcher-defined fault extension, not an official recovery benchmark. One pair cannot establish general effectiveness or statistical significance. Estimated costs are unverified against billing. Earlier connection and quota failures remain archived with null, ungraded rewards; differing protocol versions are not pooled.

Evidence: baselines/results/tau-recovery-pilot-paced-20261007/recovery-results.json under docs/research/ preserves source, protocol, grading, traces and usage. report-build-manifest.json records inputs used for this PDF.

3. Evidence of Progress (continued)

Synthetic recovery mechanics. In the seven-step fixture, step 3 incorrectly reports no remaining courses. Step 6 consumes that output and step 7 detects the problem. Selective recovery invalidates steps 3, 6 and 7 while retaining correct nodes 1, 2, 4 and 5. This illustrative nine-credit fixture is not Fisk degree requirements.

Local strategy	Recovered	Repeated	Diagnosis	Valid kept
Full restart	Yes	7	0	0
Visible-step retry	No	1	0	4
Simulated late checkpoint	No	2	0	4
Source rollback	Yes	5	1	2
Selective recovery	Yes	3	1	4

Table 2. One deterministic scenario, one execution per strategy, no model calls. Diagnosis is excluded from repeated steps. Checkpoint restoration here is a simulation, not a reproduced published checkpointing system.

Fisk source grounding. I recorded official 2024–2025 catalog facts [3], including 120 degree credits, the 2.0 GPA threshold and known major/cognate coursework. A separate partial checker calculates credits/GPA and flags missing courses, low major grades and duplicates on synthetic records.

Synthetic case	Credits	Partial audit result
Known rules met; review required	120	manual review required
Missing required course	117	requirements not met
Low major-course grade	120	requirements not met
Duplicate course attempt	120	invalid record

No case is cleared for graduation. Unresolved conditions include a lab-code inconsistency, CORE interpretation, complete prerequisites, residency, language placement and individual exceptions. Applicability to other catalog cohorts remains unverified.

Verification. October 7 verification recorded 26 primary backend test passes and one optional-environment skip, plus six adapter tests in the official environment and a successful frontend production build. Adapter tests use scripted speakers; they are software checks, not live performance trials. Later interface changes passed TypeScript and HTTP checks; signed-in visual inspection remains pending.

Code: Backend/research/benchmark.py, tau_recovery.py, official_baselines.py and fisk_case_study.py; Backend/app/research_lab.py; web/app/(app)/research/page.tsx.
Preserved evidence: docs/research/baselines/results/ and docs/research/progress-report-1/evidence/. Run IDs, traces, hashes, configurations and failed attempts are retained locally.

4. Challenges

Locating an earlier error. Retrying the visible failure can preserve corrupted state. The official-task adapter re-executes recorded reads against the same database version, finds the earliest content/error mismatch and invalidates its conservative dependency closure. Recovery restores conversation state and the isolated retail database together. This assumes deterministic trusted reads and does not establish a globally minimal recovery boundary.

Provider failures and fair comparison. Connection and quota errors interrupted early attempts. I preserved those failures and recorded a versioned amendment adding equal generation pacing to both strategies. Global attempt, time and model-usage counts include discarded work.

Access and academic ground truth. Full AgentRx diagnosis requires supported provider access. I have not claimed end-to-end reproduction. Catalog ambiguities remain flagged rather than permitting graduation clearance. Local frontend/API integration issues were addressed; browser connection problems still prevent a claim of completed visual verification.

5. Next Steps

Before the next report, I plan to complete a faithful published diagnosis/recovery baseline once access is available, validate additional fault cases, and run matched repeated trials. The saved plan defines three tasks, three faults, three seeds and six strategies (162 controlled executions), plus nine unfaulted references: **171 planned cells, not completed results**. Provider limits and costs will be assessed before execution; any scope changes will be documented first.

I will resolve Fisk catalog ambiguities, expand independent expected-result checks, and complete visual/interface verification. Comparative reporting will include fault coverage, official reward, source accuracy, repeated work, model/tool usage and cost, including unsuccessful executions.

Sources and Research Records

[1] Maintained tau benchmark, pinned official repository.
[2] AgentRx, pinned official repository; paper version.
[3] Fisk B.S. Computer Science catalog (2024–2025 edition recorded).
Local records under docs/research/: method-data-audit.md, assumptions.md, research-log.md, tau-recovery-protocol.json, fisk-catalog-rules.json and baselines/upstream-lock.json.